

Prédiction du Trafic Urbain à partir de la Structure Cartographique

Approche par Machine Learning Supervisé

Cas d'étude: Ville de New York

Données OpenStreetMap & Travel Times 2013

Albaladejo, Guilhem
Falcão Barreto, Felipe Mateus
Caramella Enzo

Résumé

Ce rapport présente le développement d'un système de prédiction du trafic routier basé exclusivement sur la structure topologique du réseau urbain et le contexte temporel. Contrairement aux approches traditionnelles qui exploitent l'historique des mesures de trafic, notre méthode repose sur l'hypothèse que la congestion est une propriété émergente de l'agencement urbain.

Indicateur	Valeur Obtenue
Volume de données traitées	~5 Go (103M observations)
Échantillons d'entraînement	2 149 844
Coefficient de détermination (R^2)	0.472
Erreur Absolue Moyenne (MAE)	7.61 km/h
Erreur Quadratique Moyenne (RMSE)	12.74 km/h

Le modèle explique 47% de la variance de la vitesse moyenne sans recourir à aucun historique de trafic. Ce résultat valide partiellement l'hypothèse que la structure urbaine encode une part significative du comportement du trafic.

1. Contexte et Problématique

1.1 Introduction

En 2026, plus de la moitié de la population mondiale vit en milieu urbain. Dans ce contexte, les métropoles et grandes agglomérations deviennent le centre d'enjeux multiples et complexes, liés aux besoins de leurs habitants, aussi bien économiques, industriels, personnels et administratifs.

Pour répondre à ces enjeux, plusieurs infrastructures complexes se superposent afin de subvenir aux besoins logistiques de ces milieux urbains. Parmi elles, l'infrastructure routière est la plus importante, puisque sur elle repose la majeure partie du transit de biens matériels mais surtout humains.

Dans ce contexte, une problématique importante émerge: pouvoir prédire le comportement des flux routiers afin d'exploiter cette infrastructure efficacement.

Dans ce but, de nombreux outils ont été développés, et sont aujourd'hui massivement utilisés: Waze et Google Maps en sont des exemples notables. Cependant, ils reposent tous sur des données collectées en temps réel, prélevées directement depuis les usagers du réseau routier, composant le trafic. Cette approche est donc limitée par la disponibilité des données, qui pourrait être affectée par la couverture internet de certaines zones routières et leur exhaustivité, tous les usagers ne souhaitant pas forcément partager leur position.

Nous proposons donc dans le suivant rapport une alternative, basée sur le postulat que si le trafic routier peut paraître à première vue chaotique, influencé aussi bien par des facteurs humains que des aléas divers, il est en grande partie une conséquence de l'infrastructure urbaine même, et qu'il est donc possible d'extraire de ses caractéristiques majeures une première approximation du trafic routier des grandes agglomérations.

Pour cela, nous utiliserons des données publiques collectées sur la ville de New York et les combinerons à des données récupérées depuis l'API publique OpenStreetMap afin d'entraîner par apprentissage supervisé un modèle capable de prédire la vitesse du trafic sur un segment de route à partir de ses caractéristiques principales et d'informations temporelles (heure de la journée, jour de semaine / weekend). L'objectif est de construire un modèle qui sera capable de généraliser ses prédictions à n'importe quelle agglomération conséquente.

1.2 Définition Formelle du Problème

Le problème se formalise comme une tâche de régression supervisée:

- Unité d'observation: segment routier (link) à un instant donné
- Variable cible Y: vitesse moyenne de circulation (km/h)
- Features X: caractéristiques structurelles du segment et contexte temporel
- Objectif: apprendre une fonction minimisant l'erreur de prédiction

2. Analyse des Données Sources

2.1 Vue d'Ensemble du Jeu de Données

Le jeu de données provient du projet de recherche sur New York. Il comprend trois tables relationnelles décrivant le réseau routier et les temps de parcours observés sur l'année 2013.

Table	Entrées	Description
links.csv	260 855	Segments routiers
nodes.csv	95 581	Intersections
travel_times_2013.csv	103 803 579	Temps de parcours

2.2 Structure Détaillée des Tables

Table links (Segments Routiers)

Chaque entrée représente un segment de route orienté entre deux intersections. Cette table constitue le graphe du réseau routier.

Champ	Type	Description
link_id	int	Identifiant unique du segment (PK)
begin_node_id	int	Intersection de départ (FK → nodes)
end_node_id	int	Intersection d'arrivée (FK → nodes)
begin_angle	float	Angle du segment vu depuis son intersection de départ.
end_angle	float	Angle du segment vu depuis son intersection d'arrivée.
street_length	float	Longueur du segment (mètres)
osm_name	string	Nom de la voie
osm_class	string	Classification OSM (motorway, residential...)
osm_way_id	int	Identifiant de la voie OSM parente
startX / startY	int	Coordonnées géographiques du noeud de départ
endX / endY	int	Coordonnées géographiques du noeud d'arrivée
osm_changeset	int	Identifie le changement d'osm qui a ajouté cette donnée
birth_timestamp / death_timestamp	int	Non-précisé dans la documentation

Table nodes (Intersections)

Chaque entrée représente une intersection ou un point remarquable du réseau routier.

Champ	Type	Description
node_id	int	Identifiant unique (PK)
xcoord / ycoord	float	Coordonnées géographiques (WGS84)
num_in_links	int	Nombre de segments entrants
num_out_links	int	Nombre de segments sortants
osm_traffic_controller	float	Non-précisé dans la documentation
birth_timestamp / death_timestamp	int	Non-précisé dans la documentation

grid_region_id	int	Identifiant dans un découpage en grille de la région urbaine de New York
----------------	-----	--

Table travel_times (Temps de Parcours)

Cette table massive contient les observations de temps de parcours agrégées par heure sur l'année 2013. C'est la source de la variable cible.

Champ	Type	Description
begin_node_id	int	Node de départ du trajet
end_node_id	int	Node d'arrivée du trajet
datetime	timestamp	Horodatage de l'observation
travel_time	float	Temps de parcours moyen (secondes)
num_trips	int	Nombre de trajets dans l'agrégation

2.3 Analyse de Qualité des Données

Une analyse exploratoire des données brutes révèle plusieurs problèmes de qualité nécessitant un traitement spécifique.

Champs redondants qui ne possèdent pas de valeurs utiles

- **birth_timestamp / death_timestamp:** valeur unique sur l'ensemble des tables, indiquant que toutes les entrées ont été créées et sont actives au même moment.
- **is_complete:** 100% des valeurs égales à **t - true** (le champ est non informatif).
- **osm_traffic_controller:** 100% de valeurs nulles sur les 95 581 nodes.
- **osm_changeset:** métadonnée OSM sans pertinence pour la prédiction.
- **Coordonnées dupliquées:** startX / startY et endX / endY dans links sont redondants avec les coordonnées des nodes référencées.

Valeur non exploitables

- **Links orphelins:** 854 begin_node_id et 872 end_node_id référencent des nodes inexistantes. Ils seront exclus dans la table nodes car ils rompent la continuité du graphe.
- **Trajets invalides:** des entrées dans travel_times avec begin_node_id et end_node_id égaux à zéro représentent des trajets sur des routes indéterminées.

Classes de Routes à Exclure

Certaines valeurs de osm_class ne correspondent pas à des routes carrossables, comme: footway (voies piétonnes), platform (quais de transport en commun), closed (routes fermées) et proposed / construction (routes en projet ou en travaux).

Ces classes représentent 22 valeurs distinctes dont seules 17 sont pertinentes pour le trafic automobile.

3. Traitement des données

Dans cette partie, nous aborderons la construction du pipeline qui nous permettra d'extraire les features précédemment déterminées à partir de nos données brutes. Pour cela nous commencerons par une analyse exploratoire de notre jeu de données, ainsi que leur nettoyage, avant de les enrichir à l'aide d'OpenStreetMap, et finalement d'en extraire des features normalisées et exploitables.

1: Nettoyage:

Comme mentionné précédemment, nos données sont constituées de 3 tables:

- Nodes.csv
- Links.csv
- Travel_times_2013.csv

1.1 Champs inutiles

Notre analyse exploratoire a révélé que certains champs ne donnent aucune information utile sur les entrées des différentes tables, nous choisissons donc de les supprimer afin d'alléger la taille de nos tables en mémoire.

1.2 Valeurs incorrectes

Notre jeu de données possède 3 types de clés identifiables:

- node_id correspondant à la clé primaire de la table node, et utilisé en tant que clé étrangère dans links et travel_times.
- link_id correspondant à la clé primaire de la table links.
- osm_way_id une clé étrangère présente dans la table links. Elle ne correspond à aucune table de notre jeu de données initial, mais le préfixe osm indique qu'il s'agit d'un identifiant utilisé par OpenStreetMap. Cela sera explicité plus tard.

Il est donc important de repérer le plus tôt possible les incohérences présentes dans les relations entre nos tables afin d'éviter d'inclure des données invalides dans notre ensemble d'entraînement.

D'après notre analyse exploratoire:

Links possède 1726 clés node_id orphelines en comptabilisant celles présentes dans begin_node_id et end_node_id. Cela pose un problème important d'intégrité des liens, puisqu'il devient impossible de les positionner dans le contexte global du graphe routier, on choisira donc de les supprimer.

Certaines lignes de `travel_times` possèdent des valeurs de `begin_node_id` et `end_node_id` nulles. L'hypothèse la plus probable est qu'il s'agisse d'une valeur attribuée par défaut aux trajets enregistrés sur des routes indéterminées. Ces lignes sont donc inexploitable.

Toutes les valeurs de `node_id` sont bien présentes dans `links`, cela indique que tous les nœuds du graphe sont au moins associés à un segment de route.

En revanche dans `travel_times`, 72000 valeurs de `node_id` ne sont pas présentes dans la table. Dans le contexte de nos features, cela correspond à des individus pour lesquels nous n'avons pas de labels. Ils sont donc inutiles pour l'apprentissage.

1.3 Champs non-exploités:

Nous pouvons également à cette étape de notre pipeline supprimer les données qui ne serviront pas à la construction de nos features, cela inclut:

Dans `nodes`:

- `osm_changeset`
- `grid_region_id`

Dans `links`:

- `osm_name`
- `osm_changeset`
- `startX`, `startY`, `endX`, `endY` (ces informations sont imputables depuis la table `nodes` et donc redondantes.)
- Dans `travel_times`:
- `num_trips`

3.2 Enrichissement

À présent que la phase de pré-traitement est terminée, nous pouvons utiliser OpenStreetMap pour récupérer les informations essentielles à la construction de nos features.

3.2.1 Création de la table `ways`

Nous avons dans la sous-partie précédente mentionné une clé `osm_way_id`. Cette clé correspond à l'identifiant des routes complètes dans l'API (contrairement à nos `links` qui correspondent à des tronçons). Nous utiliserons donc ces clés pour construire nos features relatives aux routes entières, à savoir:

- `landuse_way_dest`
- `place_way_dest`
- `max_speed`
- `lanes`

Pour cela, nous en créons une avec une entrée par clé `osm_way_id` présente dans `links`. Nous récupérons ensuite la node de destination de la “way” en identifiant le seul link la composant qui ne partage pas son `end_node_id` avec un autre segment de la “way”. Cela nous donne une destination pour environ un tiers des “ways”, certaines routes prenant la forme de rond-point ou de bifurcations, on laisse à ces routes une valeur nulle.

3.2.2 Récupération des données depuis `osmnx`

Pour effectuer des appels à l'API depuis python, les deux bibliothèques principales sont:

- `osmnx`, qui permet de récupérer des graphes entiers sur des zones géographiques précises.
- `overpy`, qui permet d'effectuer des requêtes plus ciblées depuis le `bridge Overpass`. Cependant, les serveurs utilisés par cette bibliothèque sont souvent surchargés, et donc peu fiables. C'est pourquoi nous n'avons utilisé qu'`osmnx` dans notre projet.

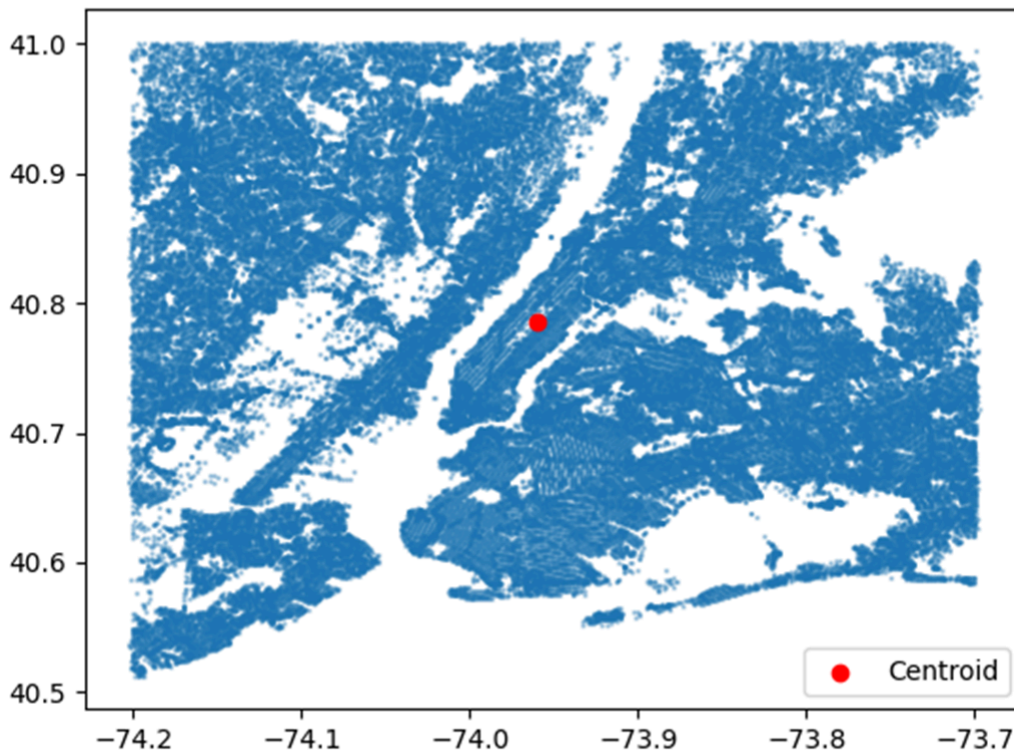
Nous créons donc un service de communication avec OpenStreetMap utilisant `osmnx`, afin de séparer les responsabilités dans notre code, incluant deux fonctions:

- `get_geodata(city: str, country: str)` renvoyant deux `GeoDataFrames` contenant respectivement les zones de `landuse` et de `place`.
- `get_ways(city: str)` renvoyant un `DataFrame` [`way_id`, `maxspeed`, `lanes`]

3.2.3 Enrichissement

Afin d'enrichir `nodes` avec les tags récupérés par `get_geodata`, il nous faut d'abord le convertir en `GeoDataFrame` afin de pouvoir faire une jointure spatiale avec ceux récupérés par la fonction. Pour cela nous utilisons les champs `xcoord` et `ycoord` de la table avec le crs `EPSG:4326` pour situer géographiquement nos intersections par latitude et longitude.

Cette conversion nous permet dans un premier temps de calculer l'emplacement du centre-ville. Pour cela on prend simplement le centroïde de notre `GeoDataFrame`. Pour la ville de New York, il se situe dans Central Park, c'est donc une approximation satisfaisante.



Localisation du centroïde (en rouge) par rapport aux nodes (en bleu).

Généré avec Matplotlib

Une fois notre table correctement convertie, il nous faut encore prendre en considération deux éléments avant de pouvoir la joindre: les zones définies par landuse et place ne couvrent pas tout l'espace, et la distance euclidienne n'est pas utilisable si l'on travaille en système de coordonnées terrestres.

Pour pouvoir solutionner ces problèmes, on crée un encodage par ordre de priorité des catégories de landuse et place. Cela nous permettra également de mieux les généraliser avec des villes utilisant des standards différents pour ces tags. On convertit ensuite nos GeoDataFrames avec le CRS relatif de la ville (pour New York c'est: EPSG: 32618). Puis on effectue une jointure spatiale par proximité.

Pour notre table ways, on effectue un simple inner join avec la table récupérée par get_ways en prenant soin de bien traiter les valeurs manquantes. On récupère ensuite les landuse et place de la destination de nos routes à l'aide des informations enrichies sur nos nodes, en prenant la node de destination si elle existe, sinon un vote à la majorité sur les nodes composant la route.

3.3 Post-traitement

Nous disposons à cette étape de toutes les données permettant de construire nos features, il nous reste juste à bien les encoder.

On calcule la centralité de chaque node comme la distance au centroïde (toujours dans le référentiel projeté de la ville). Puis on effectue une standardisation par Z-score calculé sur l'ensemble de notre distribution pour ne pas que cette feature soit liée à la taille de la ville considérée.

On calcule la feature `num_links` pour chaque node comme étant la somme de `num_in_links` et `num_out_links`.

On calcule l'angle de chaque link avec la formule: $\text{angle} = |\text{begin_angle} - \text{end_angle}|$, et on le projette dans $[0, 180]$.

On convertit les valeurs de `maxspeed` en km/h (les autres unités possibles dans OpenStreetMap sont: les mph et le nœuds (knots)). Cependant, le tag `maxspeed` d'OpenStreetMap présente des formats très variés: 50, 30 mph, 50;70, signals, walk, etc.

Comme solution, une approche a été développée: un parser robuste (`_parse_maxspeed_to_kmh`) qui extrait la première valeur numérique, détecte l'unité (mph, knots, km/h par défaut), et retourne null pour les valeurs textuelles non interprétables. Imputation des valeurs manquantes par un dictionnaire basé sur `osm_class`.

On utilise le champ `datetime` de `travel_times` pour extraire l'heure et la valeur booléenne `is_weekend`.

On calcule la vitesse mesurée sur chaque segment de route selon: $\text{speed} = \text{street_length} / \text{travel_times}$, puis on multiplie par 3,6 pour la convertir en km/h.

Nous avons agrégé les lignes `travel_times` par heure et par indicateur de week-end (`is_weekend`), en utilisant la moyenne comme critère d'agrégation pour combiner les valeurs de vitesse. Cette opération réduit la taille de la table originale de 103 millions d'entrées à environ 2 millions, un volume bien plus adapté à l'entraînement d'un modèle de random forest. Le traitement est effectué séquentiellement, avec une libération explicite de la mémoire après chaque étape du pipeline grâce à `gc.collect()`. Les jointures en bloc ont été évitées au profit de fusions incrémentales. L'agrégation finale par (`link_id`, `hour`, `is_weekend`) produit un ensemble de 2 millions d'échantillons, prêts pour l'entraînement.

On fusionne toutes nos tables pour créer une seule table contenant toutes nos features, et on y supprime nos clés `node_id`, `link_id` et `way_id`, afin d'éviter qu'elles n'influencent nos prédictions, cela nuirait à la généralisation.

On peut finalement encoder nos variables avec catégorielle à l'aide d'un *one-hot encoding* pour qu'elles puissent être exploitées par notre random forest.

Une fois ce pipeline mené à bien, il nous reste une seule table contenant nos features correctement normalisées pour être généralisables à tout type de ville et encodées pour pouvoir être utilisées par notre algorithme, ainsi que nos labels speed. On enregistre cette table dans un CSV, puis on peut passer à l'entraînement de notre modèle.

4. Description des Features

Le modèle utilise 13 features principales organisées en trois catégories: caractéristiques du segment, contexte spatial, et contexte temporel.

4.1 Caractéristiques d'une route

Feature	Type	Description et Justification
road_type	catégoriel	Classification OSM (motorway, primary, residential...). Déterminant majeur de la vitesse autorisée et du flux.
max_speed	numérique	Vitesse limite en km/h. Borne supérieure théorique de la vitesse.
lanes	numérique	Nombre de voies.
angle	numérique	Amplitude du virage (0-180°). Les virages serrés ralentissent le trafic.

4.2 Contexte Spatial

Feature	Type	Description et Justification
landuse	catégoriel	Usage du sol à l'origine. Les zones commerciales génèrent un trafic différent des zones résidentielles.
landuse_way_dest	catégoriel	Usage du sol à la destination de la way. Capture les flux directionnels.
place	ordinal	Type de zone (0-6). Hiérarchie d'urbanisation.
place_way_dest	ordinal	Type de zone à la destination.
centrality	numérique	Distance normalisée au centre. Centre-ville vs. périphérie.
num_begin_links	numérique	Connectivité à l'intersection de départ. Carrefours complexes vs. simples.
num_out_links	numérique	Connectivité à l'intersection d'arrivée.

4.3 Contexte Temporel

Feature	Type	Description et Justification
hour	numérique	Heure de la journée (0-23). Capture les heures de pointe.
is_weekend	binaire	Week-end (1) vs. semaine (0). Patterns de mobilité distincts.

Variable cible: speed, la vitesse moyenne de circulation en km/h $((\text{street_length}/\text{travel_time}) \times 3.6)$

5. Modélisation

5.1 Choix du Modèle

Une tentative initiale a été faite pour utiliser des Graph Neural Networks (GNN). L'objectif était de modéliser explicitement la propagation du trafic à travers la topologie du graphe (un embouteillage sur un segment impactant ses voisins). Cependant, cette approche s'est heurtée à une complexité technique excessive (construction de matrices d'adjacence géantes, gestion hétérogène des arêtes) et des temps de calcul prohibitifs. Cette exploration infructueuse a entraîné un retard d'une semaine sur le planning initial.

Pivot stratégique vers une approche tabulaire classique. Nous avons choisi le Random Forest Regressor. Au lieu de laisser un réseau de neurones apprendre la topologie, nous avons intégré la structure du graphe sous forme de features explicites dans le dataset (degré des nœuds, centralité, connectivité). Cette méthode s'est révélée beaucoup plus rapide à implémenter et plus robuste face aux limitations matérielles.

Ce modèle est bien adapté à ce type de données, car il permet de gérer simultanément des variables hétérogènes: catégorielles (comme **road_type**) ou continues (comme **angle**) et de modéliser des relations non linéaires. De plus, il ne nécessite pas de normalisation ou de mise à l'échelle complexe des données.

5.2 Optimisation des Hyperparamètres

Une recherche par RandomizedSearchCV avec validation croisée 5-fold a été effectuée sur l'espace suivant:

Hyperparamètre	Valeus testées	Valeur optimale retenue
n_estimators	[200, 300, 500]	200
max_depth	[none, 15, 20, 30]	30
min_samples_leaf	[1, 5, 10]	5
max_features	["sqrt", 0.5, 0.7]	0.5

bootstrap	[True, False]	false
-----------	---------------	-------

5.3 Protocole d'Évaluation

Le dataset est divisé en 80% entraînement et 20% test avec random_state=42 pour reproductibilité. Les métriques calculées sur le jeu de test sont:

- **Mean Absolute Error - MAE:** erreur moyenne en valeur absolue, interprétable directement en km/h.
- **Root Mean Squared Error - RMSE:** pénalise davantage les erreurs importantes.
- **Coefficient de détermination - R²:** proportion de variance expliquée par le modèle.

6. Résultats et Analyse

6.1 Performance du Modèle

Métrique	Valeur sur Test
Mean Absolute Error (MAE)	7.61 km/h
Root Mean Squared Error (RMSE)	12.74 km/h
Coefficient de détermination (R ²)	0.472

6.2 Distribution des Prédictions

Statistique	Vitesse prédite (km/h)
Moyenne	32.82
Médiane	30.96
Minimum	4.99
Maximum	104.74

6.3 Interprétation des Résultats

Validation de l'Hypothèse

Le R² de 0.472 signifie que 47% de la variance de la vitesse est expliquée par les seules caractéristiques structurelles et temporelles. Ce résultat valide partiellement l'hypothèse de recherche: la structure urbaine encode effectivement une part significative du comportement du trafic. Les 53% restants reflètent les facteurs non capturés: conditions météorologiques, événements ponctuels, comportement individuel des conducteurs, incidents, etc.

Précision Pratique

Une MAE de 7.61 km/h représente une erreur moyenne acceptable pour de nombreuses applications. Sur une route résidentielle à 30 km/h, cela correspond à une incertitude de $\pm 25\%$. Sur une autoroute à 100 km/h, l'incertitude tombe à $\pm 8\%$. Cette performance est remarquable considérant l'absence totale d'information sur l'historique du trafic.

Écart MAE-RMSE

Le ratio RMSE/MAE de 1.67 (contre 1.0 pour une distribution d'erreur uniforme) indique la présence d'erreurs importantes sur certains segments. Ces outliers correspondent à des situations atypiques (fermetures de route et événements) que la structure ne peut prédire.

Cohérence des Prédictions

La distribution des prédictions (moyenne 33 km/h, plage 5-105 km/h) est cohérente avec le trafic urbain new-yorkais. La proximité moyenne-médiane indique une distribution relativement symétrique, sans biais systématique.

7. Organisation et gestion de projet

7.1 Organisation

Afin de mener à bien ce projet dans le temps imparti, nous avons dans un premier défini les contraintes du projet, à savoir les features à extraire et le modèle envisagé, afin d'avoir des objectifs clairs et communs pour chaque phase du projet.

La deuxième étape a été la subdivision des tâches: nous avons défini un diagramme prévisionnel de Gantt (disponible en annexe **A3**), assignant à chaque membre du groupe une phase distincte par semaine.

Des réunions régulières ont également été organisées à la fin de chaque semaine pour partager l'état des avancements individuels.

7.2 Déroulement réel

Si notre organisation prévisionnelle était bien structurée, une charge de travail conséquente liée aux autres enseignements est une ambition initiale vue à la hausse par rapport à la contrainte de temps (celle d'utiliser un GNN) nous ont contraint à revoir l'organisation et la structure du projet dans la dernière semaine précédent le rendu.

Nous avons donc dû faire preuve d'adaptabilité pour pouvoir respecter nos objectifs avant la deadline, si cela a été une source de stress et de périodes de travail intensives qui n'étaient pas prévues dans notre organisation initiale, cela nous a également appris à réagir aux imprévus et à faire preuve de flexibilité et de rigueur dans notre organisation.

8. Conclusion et Perspectives

8.1 Bilan du Projet

Ce projet démontre la faisabilité d'une prédiction du trafic urbain basée exclusivement sur la structure cartographique et le contexte temporel. Avec un R^2 de 0.472 et une MAE de 7.61

km/h, le modèle explique près de la moitié de la variance de la vitesse sans utiliser d'historique de trafic.

L'hypothèse centrale, que le trafic est partiellement déterminé par la topologie urbaine, est donc validée. Les caractéristiques structurelles (type de route, vitesse limite, connectivité, usage du sol) combinées au contexte temporel (heure, jour de semaine) fournissent un signal prédictif significatif.

8.2 Limites de l'Approche

Variance non expliquée (53%): les facteurs exogènes (météo, incidents, événements) restent hors de portée d'une approche purement structurelle. Généralisation: le modèle est entraîné uniquement sur New York 2013, sa transférabilité à d'autres villes ou périodes n'est pas établie.

8.3 Perspectives d'Amélioration

Les futurs travaux d'amélioration se concentreront sur quatre axes majeurs: l'identification et l'intégration de features avancées pour optimiser les performances du modèle actuel; l'évaluation de modèles alternatifs, notamment les réseaux de neurones classiques (NN) ou graphiques (GNN), pour une comparaison d'efficacité; une validation approfondie et robuste via des tests de généralisation géographique (transfert entre villes) et temporelle (pérennité sur différentes années); et enfin, une analyse d'importance des features pour déterminer les variables les plus influentes dans la prédiction du trafic urbain..

8.4 Applications Potentielles

Cette approche ouvre des perspectives pour la planification urbaine (simulation d'impacts), l'estimation du trafic dans les villes non équipées de capteurs, l'établissement d'une référence structurelle pour la détection d'anomalies (baseline prédictive), et l'optimisation des itinéraires avec estimation des temps de parcours sans données temps réel.

Annexe:

A.1 Structure des Modules

Module	Fonctions Principales
pipeline.py	Orchestrateur principal, chargement CSV, appel séquentiel des étapes
pre_process.py	cleanup_data(), create_ways()
query_api.py	get_geodata(), get_ways() — interface osmnx
enrich.py	convert_to_gdf(), enrich_nodes_with_tags(), enrich_nodes_with_dist_to_center(), enrich_ways_max_speed_and_lanes()

post_process.py	post_process_nodes/ways/times/links(), create_training_table(), encode_categorical_features()
random_forest.py	Train/test split, RandomForestRegressor, évaluation MAE/RMSE/R ²

A.2 Fichiers de Données

Chemin	Description
data/raw/links.csv	Segments routiers bruts
data/raw/nodes.csv	Intersections brutes
data/raw/travel_times_2013.csv	Temps de parcours (103M lignes)
data/processed/training.csv	Dataset final (2.1M lignes)

A.3 Diagramme de Gantt

Tâche	Responsable	Semaine 1	Semaine 2	Semaine 3	Semaine 4
Phase 1: Compréhension des données et choix	Guilhem				
Phase 1: Compréhension des données et choix	Enzo				
Phase 1: Filtrage	Guilhem				
Phase 1: Compréhension des données et choix	Felipe				
Phase 2: Nettoyer données	Felipe				
Phase 2: Nettoyer données et Choix des Features	Guilhem				
Phase 3: Features, choix, réflexions et normalisation	Enzo				
Phase 4: Choix du modèle,	Felipe				
Phase 4: Compréhension regression	Felipe				
Phase 4: Implémentation du modèle	Guilhem				
Phase 5: Entraînement sur New York	Enzo				
Phase 6: Évaluation sur de New York	Guilhem				
Phase 5 et 6 + Rapport	Felipe				
Rapport	Enzo/Guilhem				